

• CRAWLER MANAGEMENT CONSOLE • DEMO WALKTHROUGH

Job-Crawler-Demo: a full-stack pipeline that turns four messy job sites into one searchable index

Not a job board. A demonstration of the plumbing behind one: **crawl heterogeneous, sometimes-hostile sites** → **normalize what they give you** → **index it for real faceted search** — the same shape of problem as any production data-ingestion pipeline.

01 · STACK

One demo, every layer of a real system

BACKEND

Node.js, Express, TypeScript

FRONTEND

Next.js, React, Feature-Sliced Design

CRAWLING

Puppeteer · Axios + Cheerio

STORAGE

PostgreSQL · Redis · Elasticsearch

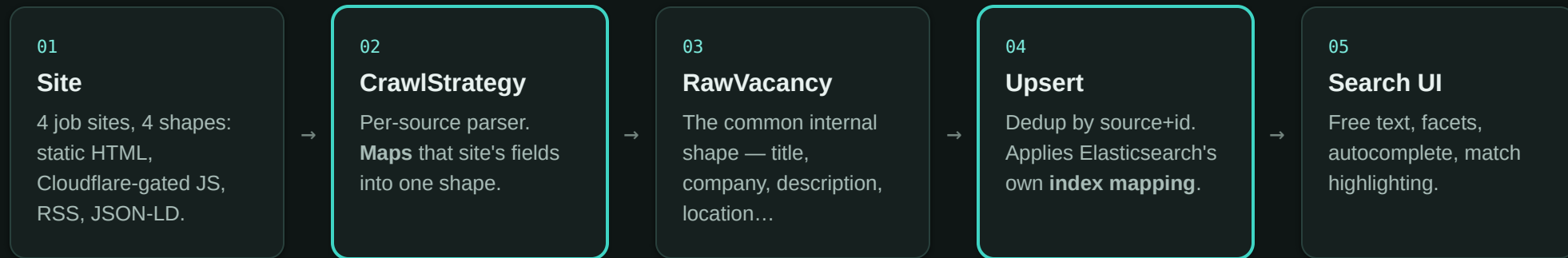
AUTH

JWT, register / login

AI (PLANNED)

Claude API enrichment

What actually happens when a source is crawled



The two highlighted stages are where "mapping" actually applies — and they mean two different things. Slide 6.

The strategy pattern doing real work

Habr Career

career.habr.com

Axios+Cheerio

Fully server-rendered — the easy case. Detail data comes from a schema.org/JobPosting JSON-LD block, more stable than scraping CSS classes.

RemoteOK

remoteok.com

Puppeteer

403s any non-browser request (Cloudflare). A real desktop UA through a real browser gets through; listing rows already embed full JSON-LD.

WeWorkRemotely

weworkremotely.com

Puppeteer + RSS

Headless Puppeteer got fingerprinted after one detail-page nav. Fix: the category's own RSS feed mirrors the listing 1:1 and isn't gated at all.

Craigslist

craigslist.org

Axios+Cheerio

No bot wall — but general classifieds, not tech. Scoped to `cat=sof` across 5 cities to match the other sources' focus.

Two different questions hiding in one word

ETL mapping

stage 02 – inside each `CrawlStrategy`

Every source's own vocabulary gets translated into `RawVacancy` at parse time, per site:

`hiringOrganization.name` → `company`

`jobLocationType === "TELECOMMUTE"` → `isRemote`

Reconciles four different worlds into one shape, **before** anything touches the database.

Elasticsearch mapping

stage 04 – the index schema

The literal ES term: field types, `keyword` vs `text` sub-fields for facets and autocomplete.

Versioned via `CRAWLER_RESULTS_SCHEMA_VERSION` — an incompatible change rebuilds the index rather than migrating in place, since it's a derived cache.

This mapping is genuinely defined **at** upsert time.

So which one was meant? "Mapping data from sources" reads as the ETL sense — which happens earlier, per source, not at the write. Both exist here; the accurate answer names both and says where each one lives.

The Elasticsearch document, end to end

```
// what one upsert actually writes – dedup key is sourceId:externalId
{
  sourceId: 6,
  externalId: "h4qi8Z4ZqSoYKXoguDQNEp", // stable per-listing id
  title: "Senior Full Stack Engineer", // text, + .suggest
  company: "Acme Corp", // text, + .keyword, + .suggest
  location: "San Francisco, CA", // text, + .keyword
  isRemote: undefined, // omitted, not false – no signal
  specialization: "Full-Stack", // keyword, best-effort
  listingId: 8, // attribution only, not the dedup key
  lastSeenAt: "2026-09-05T15:58:42Z"
}
```

What the demo is actually proving

- **One abstraction, four structurally different sites.** The `CrawlStrategy` interface never changed across static HTML, Cloudflare-gated JS, RSS, and a scoped classifieds site.
- **Real anti-bot problems, real fixes.** A live Cloudflare fingerprinting issue forced a mid-build pivot to RSS — documented as it happened, not simulated.
- **A search layer that behaves like production.** Faceted filtering, autocomplete, match highlighting, a rebuildable index.
- **Politeness by design.** Every source is rate-limited with jitter and cached — a demo scraper is still a scraper.